Sharded Blockchains

Matthieu Pigaglio

Thesis submitted in partial fulfillment of the requirements for

the Degree of Doctor in Applied Sciences

October 2026

ICTEAM

Louvain School of Engineering

Université catholique de Louvain

Louvain-la-Neuve

Belgium

Thesis Committee:

Pr. Wonderful **Supervisor** (Advisor)	UCLouvain/ICTEAM, Belgium
Pr. Good **Doctor**	UCLouvain/ICTEAM, Belgium
Pr. Another **Guy**	Another University, World

Contents

Table of Contents **i**

I Background and State of the Art **1**

1 Blockchain Fundamentals **3**

1.1 Distributed Ledgers 3

1.2 Consensus 10

1.2.1 BFT Variants 13

1.2.2 Proof-of-Work 14

1.2.3 Proof-of-Stake 16

1.3 Ethereum Proof-of-Stake 19

1.3.1 Validators 20

1.3.2 Slots and Epochs 22

1.3.3 Committees and Attestations 24

1.3.4 Fork-Choice Rule 26

1.3.5 Rewards and Safeguards 28

1.4 Transaction and State Models 31

1.4.1 UTXO Model 32

1.4.2 Account-Based Model 34

1.5 Smart Contracts and Execution Environments 38

1.5.1 The Ethereum Virtual Machine 40

1.5.2 Alternative Execution Environments 43

2 The Scalability Problem **47**

2.1 Throughput, Latency, and Full-Node Cost Bottlenecks 48

2.2 Exploiting Workload Properties 50

2.2.1 Parallel Transaction Execution 51

2.2.2 Light Nodes and Stateless Clients 52

2.3 On-Chain Scaling: Sharding 53

2.3.1 Network Sharding 55

2.3.2 Transaction Sharding 56

2.3.3 State Sharding 56

2.3.4 Existing Sharded Blockchains and Their Limitations . 59

2.4 Off-Chain Scaling: Layer 2 62

2.4.1 Rollups 63

2.4.1.1 Optimistic Rollups 64

2.4.1.2 Zero-Knowledge Rollups 65

2.4.2 State Channels 65

2.4.3 Sidechains 66

2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations 67

3 Synthesis and Motivation **71**

3.1 Open Challenges 71

3.2 Common Requirements for L1 and L2 Sharding 72

3.3 Positioning of the Contributions 73

Part I

Background and State of the Art

Blockchain Fundamentals 1

1.1 Distributed Ledgers

A distributed ledger is an append-only log of transactions, replicated by nodes that do not trust each other. Bitcoin [Nak08], launched in 2009, was the first such ledger to work in an open setting. Its transactions transfer a native currency. Ethereum [Woo14] followed in 2015 and let transactions call programs, smart contracts, whose code and storage live in the replicated state. This section covers the parts of this model that the rest of the thesis uses; how nodes agree on the content of the log is left to Section 1.2.

State machine replication. A ledger is a form of state machine replication [Sch90]. Each node holds a copy of a state S (in the simplest case, the

balance of every account), and a transaction tx updates it through a transition function, $S' = \delta(S, tx)$. Nodes that start from the same *genesis* state S_0 and apply the same transactions $tx_1, \ldots, tx_n$ all end up with

$$S_n = \delta(\ldots \delta(\delta(S_0, tx_1), tx_2) \ldots, tx_n).$$

This only works if δ is deterministic. Smart contract execution environments (Section 1.5) are built around this requirement: programs get no local clock, no floating-point arithmetic and no access to external services, since any of these could give different results on different machines. The state itself never needs to be exchanged, because any node can recompute it from the log. Nodes only have to agree on the order of transactions, and that is the job of consensus.

Blocks and hash chaining. Transactions enter the log in batches called blocks. A block has a body, the ordered list of its transactions, and a header with metadata: typically the hash of the parent header, a timestamp, and short commitments to the content of the block, such as the root of a Merkle tree over its transactions and, in some ledgers, a commitment to the state after the block is executed.

Since each header contains the hash of its parent, blocks form a hash chain back to genesis. The idea predates blockchains and was first used to timestamp digital documents [HS91]. If the hash function is collision resistant, the most recent header fixes the whole history. Altering a transaction in an old block changes that block's hash, hence the parent hash stored in its child, and so on up to the latest header. Any node that knows a recent header sees the change. Hash chaining, however, only makes a rewrite detectable. Nothing stops an attacker from building a different chain that is internally consistent,

and choosing among several valid chains is again a matter for consensus (Section 1.2).

Merkle trees. The commitments in the header are usually roots of Merkle trees [Mer87]. Leaves are hashes of data items, and each inner node hashes its children, so the root depends on every leaf. To prove that an item is in the tree, it is enough to give the sibling hashes on the path from its leaf to the root. The proof grows logarithmically with the number of leaves, and anyone who knows the root can check it. The state can be committed the same way. When the header holds the root of a tree over all state entries (account balances, contract storage), any single entry can be proven against it. How the state is laid out in practice depends on the transaction and state model of the ledger, the topic of Section 1.4.

Permissioned and permissionless ledgers. Ledgers also differ in who maintain them. In a permissioned ledger such as Hyperledger Fabric [And+18],

participants are known and authenticated in advance. Classical Byzantine fault-tolerant protocols (Section 1.2.1) then apply directly, since they assume a fixed set of n nodes of which at most f are faulty. Public blockchains such as Bitcoin are permissionless. Anyone can join or leave, and an identity is just a public key, free to generate. Nothing prevents one participant from creating many identities (a Sybil attack [Dou02]), so votes cannot be counted per identity. Voting power is tied instead to a resource that is hard to obtain: computation in proof-of-work (Section 1.2.2), locked stake in proof-of-stake (Section 1.2.3).

We only consider permissionless ledgers in this thesis. This constrains the mechanisms we add, namely the placement of state on shards in SOAPS (Part III) and the assignment of data to nodes in PANDAS (Part IV). Both must work with open membership and with some participants acting maliciously. Any node must be able to verify their outcome, and participants who would

gain from a particular outcome must not be able to bias it.

Node roles. Nodes do not all do the same work [CBC22]. A full node checks each new block by re-executing its transactions and comparing the result with the commitments in the header. It stores the current state, sometimes with a short window of recent ones. Joining nodes rarely replay the chain from genesis. They start from a recent checkpoint, download the matching state, and validate blocks from there; under proof-of-stake, the checkpoint must come from a trusted source, an issue tied to weak subjectivity (Section 1.2.3). An archive node additionally keeps every historical state, so it can answer queries such as the balance of an account at some past block, at the cost of a much larger disk footprint. Validators (miners under proof-of-work) are full nodes that also propose and vote on blocks; under proof-of-stake, they are the participants that have locked stake (Section 1.2.3). At the other end, light clients follow only block headers and check individual values with Merkle

proofs [CBC22]. They execute nothing and rely on most consensus participants being honest (Section 2.2.2).

Peer-to-peer network. Nodes form a peer-to-peer overlay in which each keeps a bounded number of connections. Transactions and blocks spread by gossip: a node that receives a valid message forwards it to some of its neighbors [Dem+87]. Peers find each other through a discovery protocol, often a distributed hash table such as Kademlia [MM02]. To keep bandwidth in check, practical gossip protocols push full messages to a few neighbors and only announce them, with short metadata, to the rest.

The network matters for security. How fast blocks propagate bounds how fast they can be produced before security degrades [GKL15; Ger+16], which is why block intervals are set well above the propagation delay.

Full replication. In the model so far, every full node stores the whole state and re-executes every transaction of every block. No node has to trust an-

other, but adding nodes adds no capacity. Throughput is capped by what one node, run by an ordinary participant, can download, store and execute. Larger or more frequent blocks would lift the cap, at a price. Running a node would become more expensive, so fewer participants could afford it, and blocks would propagate more slowly, which means more forks and weaker security [Ger+16]. Chapter 2 discusses this problem and the main ways around it. This thesis builds on one of them, sharding, where each node handles only a subset of the transactions and of the data.

1.2 Consensus

Nodes that apply the same transactions in the same order reach the same state (Section 1.1). A consensus protocol makes them agree on that order, block after block, even when some nodes are faulty or malicious and the network delays messages [LSP82].

Fault and network models. Blockchain consensus protocols generally assume *Byzantine* faults: a faulty node may do anything. It may stop responding, send wrong messages, collude with other faulty nodes, or *equivocate*, i.e., send conflicting messages to different nodes, such as votes for two different blocks at the same height. Assumptions about the network vary more. The synchronous model guarantees delivery within a known bound Δ. The asynchronous model gives no bound at all, and Fischer, Lynch, and Paterson proved that in this model no deterministic protocol can guarantee agreement if even a single node may crash [FLP85]. Partial synchrony sits in between. The network may be asynchronous for a while but eventually delivers messages within a bound, and agreement is then possible with fewer than a third of Byzantine nodes [DLS88]. Longest-chain protocols assume synchrony [GKL15]; most BFT protocols target partial synchrony [CL99; Yin+19].

Safety and liveness. Two properties are expected from consensus. *Safety* means that honest nodes never confirm two conflicting blocks, so a confirmed transaction is never reverted. *Liveness* means that the protocol keeps making progress, so a valid transaction from an honest client is eventually included. When the network can be partitioned, the two pull against each other. A protocol that keeps producing blocks on both sides of a partition cannot also promise that those blocks never conflict. One answer is to run two layers: a chain that keeps growing under any conditions, and a slower finality mechanism that confirms a prefix of it only once enough participants agree [NTT21]. Ethereum is built this way (Section 1.3).

Sybil resistance. Because votes in a permissionless ledger are weighted by a costly resource and not counted per identity (Section 1.1), security assumptions are stated in terms of that resource: honest participants hold a majority of the computing power in proof-of-work (Section 1.2.2), or more than two

thirds of the stake in proof-of-stake (Section 1.2.3). Classical BFT protocols (Section 1.2.1) offer no Sybil resistance on their own. In practice they are often run on top of proof-of-stake, by a committee chosen according to stake.

1.2.1 BFT Variants

Classical Byzantine fault-tolerant (BFT) protocols, from PBFT [CL99] onward, assume a fixed, known set of n nodes, at most f of them Byzantine, with $n \geq 3f + 1$. Decisions need a *quorum* of $2f + 1$ nodes. Two quorums overlap in at least $f + 1$ nodes, hence in at least one honest node, and an honest node never votes for two conflicting values. Two conflicting decisions can thus never both reach a quorum. In PBFT, a leader proposes a value and the nodes agree on it in three phases: pre-prepare, prepare, and commit. A faulty or slow leader is replaced through a *view change* [CL99]. Unlike in longest-chain protocols, finality is immediate: a decision is never reverted as long as the bound on f holds.

The cost is communication. PBFT has every node send its votes to every other node, $O(n^2)$ messages per decision, which in practice limits the number of participants. HotStuff [Yin+19] brings this down to $O(n)$: the leader collects the votes and combines them into one threshold signature. Aggregatable signatures such as BLS [BLS01] serve the same purpose, turning thousands of votes into a single signature checked in one operation.

Sharded blockchains such as OmniLedger [Kok+18] and RapidChain [ZMR18] run a BFT protocol inside each shard, among the committee assigned to it. Committee size is then a trade-off. Small committees agree faster, but they are more likely, by chance, to have more than a third of malicious members. Section 2.3 comes back to this.

1.2.2 Proof-of-Work

Proof-of-work was first proposed against spam, as a small computation required for each email [DN92]. Bitcoin turned it into a way to elect who ap-

pends the next block [Nak08]. A miner produces a valid block by finding a nonce for which the hash of the block header falls below a target. No shortcut is known, so miners try values one by one, and the chance of finding the next block is proportional to one's share of the total computing power. The target is adjusted periodically to keep the block rate roughly constant, at one block every ten minutes on average in Bitcoin.

When two miners find a block at nearly the same time, the chain splits into two branches, or *forks*. A *fork-choice rule* decides which branch to follow. Bitcoin's is the longest chain rule (more precisely, the chain with the most accumulated work), and miners build on the branch it selects. GHOST [SZ15] follows the heaviest subtree instead. Blocks on discarded branches then still count, and the protocol stays secure at higher block rates. Proof-of-stake fork-choice rules later built on this idea (Section 1.3.4).

Finality under these rules is only probabilistic. A heavier branch can al-

ways revert a block, but the probability drops exponentially with the number of blocks on top of it, hence the usual advice to wait for several confirmations. The Bitcoin backbone analysis [GKL15] proves the protocol secure when honest miners hold a majority of the computing power and blocks propagate quickly relative to the block interval. Larger or more frequent blocks cause more forks, and the work spent on discarded branches no longer protects the chain [Ger+16]. Throughput cannot simply be raised by tuning these parameters, and Chapter 2 starts from this observation. Proof-of-work is also energy-hungry, since security grows with the computation miners spend.

1.2.3 Proof-of-Stake

In proof-of-stake, participants called *validators* lock an amount of currency, their *stake*, and their weight in the protocol is proportional to it. They earn rewards for proposing blocks and voting correctly. Ouroboros [Kia+17] and Algorand [Gil+17] both come with formal security proofs, but take different

routes. Ouroboros keeps a longest-chain structure and picks block producers according to stake. Algorand draws small committees by cryptographic sortition, and each committee decides on a block with a Byzantine agreement protocol.

Since the protocol holds the stake, it can punish misbehavior directly. A validator that equivocates, for instance by signing two different blocks for the same position, produces two signed messages that together prove the fault and identify who committed it [She+21]. Anyone can put this proof in a block, and the protocol then destroys part of the validator's stake. This is called *slashing*. Security rests on economics: an attack needs a large share of the stake and puts that stake at risk. SOAPS (Part III) relies on rewards and slashing, and Section 1.3.5 describes how Ethereum implements them.

At each step, a proof-of-stake protocol must pick the proposer of the next block and the committees that vote on it, using randomness produced by the

protocol itself [Gil+17]. If this randomness is predictable long in advance, attackers can prepare against the selected nodes. If participants can influence it, they can steer the selection toward positions that benefit them, and this has been shown to work in practice [Nag+25]. SOAPS and PANDAS inherit the problem, since both assign tasks to nodes from this randomness (Section 1.3.3).

Two further problems are specific to proof-of-stake. Signing a block costs nothing, so a validator could vote on every competing branch; slashing exists to rule out this *nothing-at-stake* behavior. Slashing has a blind spot, though. Once validators have withdrawn, their old keys can still sign an alternative history from far in the past (a *long-range attack*). A node joining the network cannot tell such a history from the real one using the genesis block alone, and needs a recent checkpoint from a source it trusts, an assumption called *weak subjectivity*.

1.3 Ethereum Proof-of-Stake

All three contributions target Ethereum, and two of them depend directly on its consensus. SOAPS (Part III) builds on its committees and penalties, and PANDAS (Part IV) must fit in the timing of its slots. This section goes into more detail than Section 1.2 for that reason. Unless stated otherwise, parameters come from the consensus specifications [Eth].

Ethereum launched with proof-of-work and moved to proof-of-stake in September 2022, in the upgrade known as *the Merge*. Since then, an Ethereum node runs two programs. The *consensus client* runs the *beacon chain*: it manages the validator set, assigns validators to committees, and decides which blocks belong to the chain and which are final [Sch+22]. The *execution client* holds the state and the pool of pending transactions, and executes the transactions of each block in the Ethereum Virtual Machine (Section 1.5.1). The two talk over a local interface, the Engine API, through which the consensus

client asks the execution client to build or check the *execution payload*, the list of transactions carried by a beacon block. The beacon chain itself is inherited from earlier sharding designs, where it was meant to coordinate many shard chains (Section 2.3). The system model of SOAPS follows that original design.

1.3.1 Validators

Becoming a validator requires a deposit of at least 32 ETH in a deposit contract and running both a consensus and an execution client. Each validator has an *effective balance*, derived from its actual balance, which sets its voting weight and its chance of being picked as proposer or aggregator. Until the Pectra upgrade in May 2025, the effective balance was capped at 32 ETH, which forced large stakeholders to run many validators. Pectra raised the cap to 2048 ETH for validators that opt into compounding withdrawal credentials, so stake can now be consolidated into fewer validators. Stake remains concentrated all the same: staking pools and liquid staking services control a

large share of it, spread over many separate validators.

Validators have three regular duties. When selected as the proposer of a slot, they propose a block. Once per epoch, they *attest*, i.e., they sign a vote on the current head of the chain and on the latest checkpoints (Section 1.3.3). Some are also chosen as aggregators and combine the attestations of their committee into one signature. Since the Altair upgrade, there is also a *sync committee* of 512 validators, renewed every 256 epochs (about 27 hours), which signs each block header. Light clients can then follow the chain without tracking the whole validator set (Section 2.2.2).

Validators enter and leave the active set through queues. A *churn limit* bounds how many can be activated or exit per epoch. The validator set cannot change too quickly, and in particular an attacker cannot gain a large weight in a short time. The active set is on the order of a million validators. At this scale, a classical BFT protocol in which every validator exchanges votes with

every other (Section 1.2.1) is out of the question. Ethereum relies instead on committees and signature aggregation, described below.

1.3.2 Slots and Epochs

Time is divided into *slots* of 12 seconds, and 32 slots make an *epoch* of 6.4 minutes (Figure 1.1). Each slot has a single proposer, which may publish one block. If it does not, the slot stays empty.

Proposers are drawn at random, weighted by effective balance, from a value called the RANDAO mix. Every proposer includes in its block a signature over the current epoch number, and this signature is mixed into the RANDAO value. The mix at the end of an epoch seeds the selection of proposers and committees for a later epoch. A proposer cannot choose its contribution, since its signature is fixed in advance. It can withhold its block, though, and its contribution is then skipped. The last proposers of an epoch can exploit this to pick among several possible seeds and bias future proposer selection in

their favor [Nag+25]. The selection is also public before the slots it concerns. Nodes can prepare, but so can an attacker, who knows which node to target at what time (e.g., with a denial-of-service attack).

Each slot follows a fixed schedule. At 0 s, the proposer publishes its block, which then propagates through the network. The validators in the committee of the slot attest to the head of the chain as soon as they have received and validated the block, and at the latest at 4 s. At 8 s, aggregators publish the aggregated attestations of their committee. A validator thus has at most 4 seconds to receive and check a block before voting. If the block arrives late or cannot be checked in time, the validator votes for the previous head, and the block will likely be left out of the chain. In PANDAS (Part IV), validators must also verify that the data attached to the block is available, and this check has to fit in the same 4 seconds.

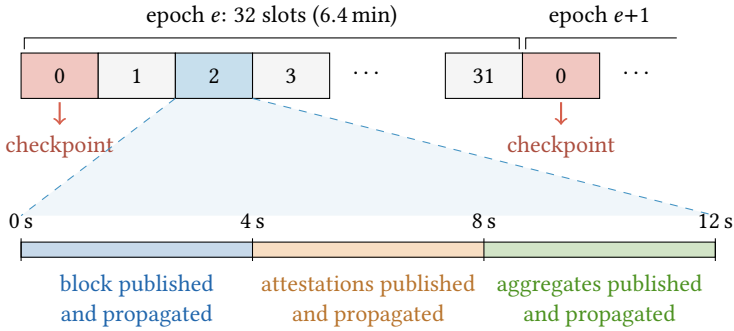


Figure 1.1: Slots and epochs in Ethereum proof-of-stake. An epoch has 32 slots of 12 seconds, and the first slot of each epoch serves as its checkpoint. Within a slot, the block is published at 0 s, attestations at 4 s, and aggregated attestations at 8 s.

1.3.3 Committees and Attestations

At each epoch, the active validators are shuffled with the RANDAO seed and split into committees. Each slot gets between 1 and 64 committees, and each validator sits in exactly one committee per epoch, so it attests exactly once per epoch. When there are enough validators, the shuffle keeps every committee at or above 128 members, above the minimum of 111 recommended for committee safety. With around a million validators, committees are in fact several times larger, with several hundred members each.

The shuffle is deterministic and uses only public data, the seed and the

list of active validators. Any node can recompute the committees and check that an attestation comes from a member of the right one, without trusting anybody. Our two designs reuse this pattern of a deterministic function of public randomness that any node can recompute. SOAPS uses it to elect the validator that computes the placement of state on shards, and the witnesses that check it; PANDAS uses it to decide which node is responsible for which part of the data.

An attestation carries three votes. The *head vote* names the block that the validator sees as the head of the chain, and feeds the LMD-GHOST fork-choice rule. The *source* and *target* votes name two checkpoints, and feed the Casper FFG finality mechanism [Sch+22]. Attestations use BLS signatures [BLS01], which can be aggregated: the signatures of all committee members who cast the same vote collapse into one, with a bit field recording who signed. Without aggregation, about a million signatures per epoch would

have to go into blocks, more than the bandwidth of nodes can carry.

1.3.4 Fork-Choice Rule

The consensus protocol of Ethereum, Gasper, combines two mechanisms [Sch+22; NTT21]. LMD-GHOST, a fork-choice rule, selects the head of the chain and lets it grow even when many validators are offline. Casper FFG, a finality mechanism, runs on top of this chain and finalizes some of its blocks. This is the ebb-and-flow structure of Section 1.2: the chain selected by LMD-GHOST stays available, while the finalized prefix stays safe under network partitions [NTT21].

LMD-GHOST adapts GHOST (Section 1.2.2) to votes. Starting from the latest justified checkpoint, it follows at each fork the branch whose subtree carries the most weight, counting only the latest attestation of each validator (hence *latest message driven*).

Casper FFG reasons in epochs, and the checkpoint of an epoch is the block

at its first slot. Once validators holding at least two thirds of the stake attest with the same source and target, the target checkpoint is *justified*. A justified checkpoint is *finalized* when the checkpoint of the next epoch is justified with it as source. Under normal conditions, a block is final after two to three epochs, about 13 to 19 minutes. Reverting a finalized block would mean finalizing two conflicting checkpoints. That takes at least a third of the stake signing contradictory votes, which can be proven and slashed (Section 1.3.5).

The fork-choice rule has been attacked in several ways. In a balancing attack, a few Byzantine validators time the release of their votes so that honest validators stay split between two branches. In an ex-ante reorganization, the attacker withholds a block and its votes, and releases them later to outweigh the next honest block [Sch+22]. The protocol now counters both with *proposer boost*. A block received on time, before the 4 s mark, gets extra weight in the fork choice until the end of the slot, equal to 40% of the weight of a

committee.

Validators should vote only for blocks that are valid and whose data is *available*, meaning that anyone can download it. With Data Availability Sampling (Section 2.4), they check availability by downloading a few random samples of the data rather than all of it. The *tight* fork-choice rule has them run this check before attesting, so a block whose data cannot be retrieved never becomes the head, however valid it is otherwise. The *trailing* rule runs the check later. Validators get more time, but the guarantee is weaker, since others may build on a block before its data has been checked. PANDAS targets the tight rule, which is why sampling must finish within the first 4 seconds of the slot [Pig+25].

1.3.5 Rewards and Safeguards

Validators are paid for performing their duties correctly and on time, and lose a little when they miss them. The reward for an attestation depends on

which of its three votes are correct and on how quickly it was included in a block. Proposers earn rewards for including attestations and evidence of misbehavior, and sync committee members for signing headers.

A validator is *slashed* for signing messages that contradict each other: two different blocks for the same slot, two attestations with the same target epoch but different content, or an attestation whose source and target surround those of another of its own [Sch+22]. The proposer that includes the resulting proof (Section 1.2.3) receives a reward. The slashed validator immediately loses 1/4096 of its effective balance, is forced to exit, and can withdraw what remains only after 8192 epochs, about 36 days. Halfway through this period, it pays a *correlation penalty*: the fraction of the total stake slashed in the same window, multiplied by three, is taken from its balance, up to the whole of it. A one-off fault, usually a misconfiguration, costs little. A coordinated attack by a third of the stake costs the attackers all of it.

Another safeguard covers the case where finality stops. With more than a third of the stake offline, no checkpoint can gather two thirds of the votes, and the chain stops finalizing. After four epochs without finality, the *inactivity leak* kicks in. Validators that do not participate lose balance at an increasing rate, until those still active again hold two thirds of the stake and finality resumes. The leak has to be designed with care: Pavloff et al. show that Byzantine validators can use it to drain the stake of honest validators and raise their own relative weight [PAT24].

Ethereum, then, does not prevent misbehavior. It punishes it whenever it can be proven and attributed to a validator. SOAPS relies on these same mechanisms (election, stake, and slashing) instead of adding a consensus protocol of its own. Every deviation from SOAPS must therefore be detectable and attributable to a specific validator, which is the purpose of its challenge mechanism.

Finally, the block of a slot is often not built by its proposer. The order of transactions in a block can be profitable, for example when one's own transaction is placed just before or after a large trade. This profit is called *maximal extractable value* (MEV) [Dai+20]. Capturing it takes specialized infrastructure, and Ethereum has adopted, outside the protocol, a form of *proposer-builder separation* (PBS). *Builders* assemble blocks and bid for them to be proposed; *relays* pass the highest bid to the proposer, which signs the header without seeing the content. A handful of builders produce most blocks [Hei+23]. For PANDAS, this means that the data to distribute and sample starts at a few well-connected, well-provisioned builders, while most of the nodes sampling it run on modest hardware.

1.4 Transaction and State Models

The state machine view of Section 1.1 says nothing about what the state contains, or how a transaction designates the part of it that it reads and mod-

ifies. That is the role of the state model. It defines which data a transaction accesses, and so when two transactions conflict, which in turn governs whether they can run in parallel (Section 2.2.1) and how the state can be split between shards (Section 2.3.3). Bitcoin introduced the *unspent transaction output* (UTXO) model [Nak08], in which the state is a set of coins. Ethereum uses the *account-based* model [Woo14], in which it is a set of accounts with balances and storage. We work with accounts throughout, but start with UTXOs, because the contrast explains several difficulties that Parts II and III deal with.

1.4.1 UTXO Model

In the UTXO model, the state is the set of transaction outputs not yet spent. An output holds an amount and a spending condition, usually a signature from the key of its owner. A transaction consumes existing outputs, its *inputs*, and creates new ones. Each input names the output it spends, by the identifier of the transaction that created it and its index there, and supplies the data that

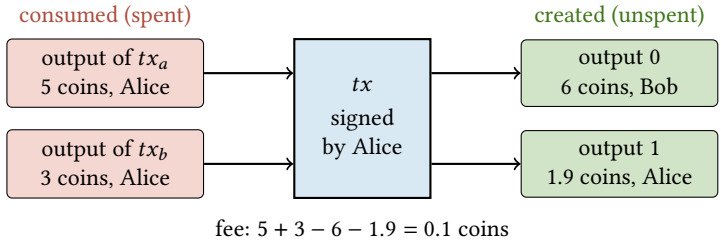


Figure 1.2: A transaction in the UTXO model. It consumes two outputs owned by Alice, which are removed from the set of unspent outputs, and creates two new ones: a payment to Bob and the change returned to Alice. The difference between inputs and outputs is the fee.

satisfies the spending condition. Outputs cannot exceed inputs in total, and the difference goes to the block producer as a fee (Figure 1.2). An output can be spent only once. Two transactions spending the same output form a *double spend*, and at most one of them makes it into the ledger.

Because inputs are listed in the transaction, the state a transaction depends on is known before it runs. Two transactions conflict only if they spend the same output, or if one spends an output that the other creates. Transactions with no outputs in common can be validated in parallel. A sharded design can place each output on a shard, say by the identifier of the transaction that created it, and know in advance which shards a transaction touches.

Several of the first sharded blockchains were built on UTXOs for this reason [Kok+18; ZMR18], and transaction placement methods have been proposed for the model [Ngu+19].

The catch is that outputs are consumed whole and hold no persistent data. Applications with shared, long-lived state, such as a token or an exchange used by many users, are hard to express. Extended UTXO models attach data and more expressive scripts to outputs while keeping explicit inputs, but the application still has to split its state into outputs that are consumed and recreated on every update. The workloads we study and the systems we target use accounts, so we leave UTXOs aside from here on.

1.4.2 Account-Based Model

In the account-based model, the state maps addresses to accounts. Ethereum has two kinds of accounts. *Externally owned accounts* (EOA) are controlled by a private key and hold a balance and a *nonce*, the number of transactions

the account has sent. Every transaction carries the nonce of its sender. This forces the transactions of an account to execute in order, and keeps a signed transaction from being replayed. *Smart contract accounts* (SCA) are created by deploying code. Besides a balance and a nonce, they hold that code and a persistent *storage*: a mapping from 256-bit keys, called slots, to 256-bit values, which only the code of the contract can modify (Section 1.5). A token contract typically keeps the balance of each holder in its own storage slot.

Transactions are sent by EOAs. A transaction transfers currency to another account, deploys a new contract, or calls an existing one with some input data. The called contract may call others, which may call others again, so one transaction can read and modify many accounts and storage slots. Only the original transaction appears in the block. The *internal transactions* it triggers can only be observed by executing it (Section 1.5.1) [PKR23]. Activity is also very uneven: a small number of accounts, such as exchanges, popular

token contracts and decentralized finance protocols, take part in a large share of all transactions [GDW19].

The key difference from UTXOs is that the state a transaction will access is not known in advance. It depends on the code of the contracts involved and on the current contents of their storage, which may change between the moment the transaction is signed and the moment it runs. The same call can touch different slots, or even different contracts, depending on what executed before it. Whether two transactions conflict, i.e., whether one writes data that the other reads or writes, is generally known only after both have run. Ethereum does let a transaction declare an *access list* of the accounts and slots it plans to use. The list is optional, though, and the transaction may still touch state it did not list. This makes parallel execution (Section 2.2.1) and state sharding (Section 2.3.3) harder with accounts than with UTXOs. Part II measures how real transactions spread their accesses over time and across one

another, and the same uncertainty complicates the placement of accounts in SOAPS (Part III).

Executing a transaction costs *gas*. Each operation of the execution environment has a fixed gas cost reflecting the resources it uses, and storage reads and writes are among the most expensive. The sender sets a maximum amount of gas and pays for the gas actually used. Since the London upgrade, the gas price has two parts [Rou24]. A *base fee*, set by the protocol, rises when blocks are fuller than a target, falls when they are emptier, and is burned. A *priority fee*, chosen by the sender, goes to the proposer. When demand is high, it decides which transactions get in first, and so how long a transaction waits to be included [Pac+23]. Each block also has a *gas limit* that bounds its total computation, and this limit is, in effect, the throughput of the chain. It is kept low enough for nodes on modest hardware to execute a block within a slot, a direct consequence of full replication (Section 1.1).

1.5 Smart Contracts and Execution Environments

A *smart contract* is a program stored in the ledger and executed by every full node whenever a transaction calls it. Its state, such as balances or open positions, is part of the replicated state, and only the code of the contract can modify it. All nodes run the same code on the same state and get the same result, so nobody has to be trusted to execute the contract correctly.

Much of the activity on Ethereum goes through a few kinds of contracts. Token contracts record who owns which units of a fungible asset (the ERC-20 interface) or which non-fungible items (ERC-721). Around them sit decentralized exchanges, which let users swap one token for another against liquidity pools held by a contract, lending protocols, which lend tokens against collateral, and bridges, which lock assets on one chain to release equivalent assets on another. In the workload analyzed in Part II, transactions involving contracts took over as time went on, and simple transfers, which made up more

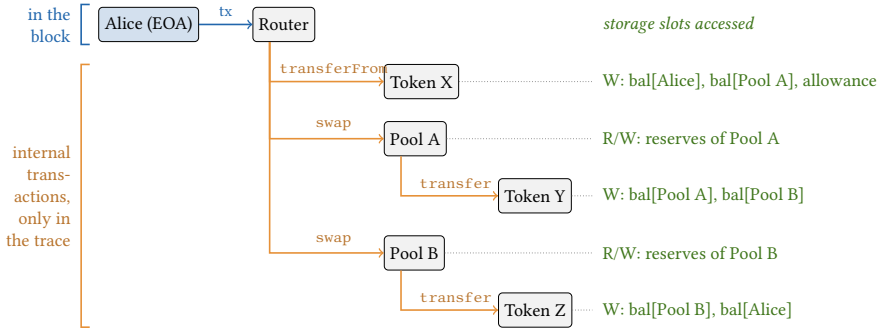


Figure 1.3: Calls made by a transaction in which Alice swaps token X for token Z through two exchange pools (X to Y in Pool A, then Y to Z in Pool B). Only the transaction from Alice to the router is recorded in the block. The internal transactions, and the storage slots that each contract reads (R) and writes (W), are only visible by executing the transaction.

than 80% of early transactions, have become marginal [PKR23].

Contracts call each other, and applications are often assembled from existing contracts. A swap on a decentralized exchange may go through a router contract, then through one or more pools, each calling the token contracts of the assets it holds, so a single transaction can end up involving tens of contracts. Even the simplified two-step swap of Figure 1.3 involves six.

These calls are what create dependencies between accounts in the account-based model (Section 1.4.2). They also explain why some contracts, widely used tokens and exchange pools in particular, are accessed by a large frac-

tion of all transactions. Such contracts turn into contention points as soon as transactions run in parallel or the state is split between shards.

1.5.1 The Ethereum Virtual Machine

Ethereum contracts run on the Ethereum Virtual Machine (EVM) [Woo14], a stack machine that operates on 256-bit words. During a call, a contract sees a stack, a temporary *memory* discarded when the call ends, the input data of the call, and its persistent *storage* (Section 1.4.2). Every instruction costs gas, and execution aborts with an error when the gas of the transaction runs out, so every execution terminates. Storage accesses are by far the most expensive instructions. Reading a slot for the first time in a transaction costs hundreds of times more than an arithmetic operation, and writing a slot that was previously zero costs more still, because each such operation reads or updates the state database of every node.

A contract calls another with CALL, which runs the code of the callee on

the storage of the callee. STATICCALL does the same but forbids any change to the state. DELEGATECALL runs the code of the callee on the storage of the caller, which is how libraries and upgradable contracts work. When a call fails, its changes to the state are reverted, and the caller is free to continue and handle the failure. Transactions in a block run one after the other, in the order chosen by the block producer, each seeing the state left by the previous one.

The state is stored in Merkle Patricia Tries, which combine the Merkle trees of Section 1.1 with a prefix tree (Patricia trie), so that each key determines a unique path from the root to its value. There are two levels. The *world-state trie* maps the hash of each address to its account: nonce, balance, code hash, and storage root. Each contract has its own *storage trie*, which maps the hash of each slot to its value. The root of the world-state trie is the state root found in each block header. Reading a value means walking from

the root down to a leaf, which may take several disk accesses. Because keys are hashed, entries that belong together in the application, two slots of the same contract for instance, land in unrelated parts of the trie and of the disk. State access is one of the main costs of executing a block, and clients cut it down with caches and flat copies of the latest state. Those caches only pay off if transactions tend to access state that was accessed recently, a property called temporal locality (Section 2.2).

The block alone does not reveal the read and write sets of a transaction, since internal transactions are not recorded in it (Section 1.4.2). To get them, we replay the whole chain with the Go Ethereum (geth) client and record, for every transaction, the accounts it reads and writes, including those touched by its internal transactions (Figure 1.3). Replaying the chain up to the Merge takes about three days and yields a dataset of about 2 TB; Part II describes the tool we built to process it [PKR23].

Sequential execution over a single global state, together with a gas limit that keeps execution within reach of modest nodes (Section 1.4.2), caps the throughput of Ethereum at roughly 15 to 30 tx/s.

1.5.2 Alternative Execution Environments

Many other blockchains reuse the EVM, as do most layer-2 protocols (Section 2.4). EVM compatibility lets them run existing contracts and reuse the same development tools, and their workloads tend to look like that of Ethereum, because they host the same kinds of applications. Chains with other virtual machines, for example based on WebAssembly, usually keep the same model of contracts executed sequentially over shared state.

Another line of work changes the execution model itself to allow parallel execution. Dickerson et al. proposed that block producers execute contract calls speculatively in parallel and publish the resulting schedule, so that validators can replay the block in parallel and deterministically [Dic+17]. Sev-

eral recent blockchains were designed for parallel execution from the start. Solana has each transaction declare the accounts it will read and write, and runs in parallel the transactions whose accounts do not overlap. Sui models the state as objects, and separates objects owned by a single user from shared ones. Transactions that use only owned objects need no ordering relative to others and skip consensus; only those on shared objects go through it [Bla+24]. Aptos uses Block-STM, which runs the transactions of a block optimistically in parallel, detects conflicts at runtime, and re-executes the transactions involved, giving the same result as a sequential execution in block order [Gel+23].

These designs take one of two approaches. Either transactions declare their dependencies up front, which comes naturally with UTXOs but constrains how contracts are written with accounts, or the system executes optimistically and bets on conflicts being rare. Either way, the gain depends on

the workload, since transactions that access the same accounts still have to run one after the other. Section 2.2 returns to these techniques.

The Scalability Problem 2

- Here I explain why blockchains do not scale, present the two families of solutions (sharding on L1, and L2s), and for each one say clearly what is still missing.

- Much of this can come from the related work sections of the locality, SOAPS and PANDAS papers, extended.

- Order: bottlenecks, optimizations that rely on the workload, sharding, L2 and data availability, and the limitations of each.

- At the end, a table comparing existing approaches on a few criteria: what gets sharded, whether it uses the workload, whether assignments can be verified, whether it is part of consensus, and whether it was

evaluated on a real workload.

2.1 Throughput, Latency, and Full-Node Cost Bottlenecks

- Throughput is limited because every node re-runs every transaction. Compare with centralized payment systems to give an order of magnitude [Ger+16].
  - Bitcoin and Ethereum L1 handle tens of transactions per second, card networks thousands.
  - Bigger blocks or a higher gas limit mean more expensive nodes and slower propagation [Ger+16].
- Latency depends on the block interval and on how long finality takes, both limited by propagation and consensus rounds.
  - On Ethereum a transaction gets in within a 12 s slot, and is final after about two epochs, so around 13 minutes [Pac+23].

- Running a full node costs storage (the state keeps growing), bandwidth (blocks and data) and CPU. If these go up, small participants drop out and the network becomes less decentralized [Ger+16].
  - The state already holds hundreds of millions of accounts, and the history is much bigger.
  - For publishing data, bandwidth is what runs out first. This is the motivation for Part IV.
- I can mention the trilemma here, but only briefly, as context. It is not what drives the thesis.
- The message I want to get across: L1 and L2 both need to scale, and the general answer is sharding, meaning that no single node handles every transaction or every piece of data.
  - Sharding computation means not every node executes every transaction (Part III).

  - Sharding data means not every node downloads every piece of data (Part IV).

2.2 Exploiting Workload Properties

- Some optimizations leave the protocol alone and instead count on the workload behaving well: few conflicts, and state that gets reused.

- I introduce them here because Part II checks whether those assumptions actually hold on Ethereum.

- Introduce the two kinds of locality informally (Part II gives the formal definitions):

  - spatial locality, or disjoint access parallelism: transactions running at the same time touch different state;

  - temporal locality: state used recently is likely to be used again soon.

2.2.1 Parallel Transaction Execution

- The idea is to run transactions that do not conflict at the same time inside a block.
  - The result must still be the same as running them one by one in block order.
- Three ways to do it: declare access lists up front (EIP-2930, Solana), run optimistically and redo on conflict (Block-STM [Gel+23]), or have the proposer compute a schedule (Dickerson et al. [Dic+17]).
  - With declared access lists, declaring too much loses parallelism and declaring too little makes the transaction fail.
  - The optimistic approach needs nothing from users, but it slows down when there are many conflicts [Gel+23].
  - In Dickerson et al., the proposer runs the block speculatively and

publishes a schedule that validators can replay in parallel [Dic+17].

- How well any of this works depends on how many transactions conflict, which is the spatial locality measured in Part II.

- I do not contribute to parallel execution myself. I mention it because Part II's results say something about it, and I come back to it only as a perspective.

2.2.2 Light Nodes and Stateless Clients

- Light clients check headers and Merkle proofs instead of keeping the whole state [CBC22].

  - They trust that the majority of the consensus (in Ethereum, of the sync committee) is honest. On their own they cannot spot an invalid block or missing data [CBC22].

  - Fraud proofs and data availability sampling were proposed to fix

that [Al-+21]. This leads into Section 2.4.4.

- Stateless clients go further: blocks come with witnesses for the state they touch, and caching recently used state makes those witnesses smaller.
  - Witness size costs bandwidth, and the trie design (Merkle Patricia vs. Verkle or binary trees) changes how big the proofs are.
- How well this works depends on how often state is reused, which is the temporal locality measured in Part II.

2.3 On-Chain Scaling: Sharding

- The principle: split nodes, state and transaction processing over several chains running in parallel (shards), each with its own committee, and coordinate them through a beacon chain [Wan+19].
  - In the ideal case, throughput grows linearly with the number of shards.

  - The beacon chain keeps the validator list and the randomness, assigns validators to shards, and records checkpoints (cross-links) of shard blocks.

- Security: a small committee is easier to take over, which is why nodes are assigned at random and reshuffled regularly [Ava+23].

  - Show how the chance of a corrupted committee depends on committee size and on the overall share of malicious nodes (hypergeometric tail). Committees are made big enough for this to be negligible.

  - Avarikioti et al. give a formal definition of what robust sharding should guarantee, including against adaptive adversaries [Ava+23].

- Following Wang et al. [Wan+19], split the discussion into network, transaction and state sharding.

- Reuse the SOAPS figure that shows a sharded blockchain with an intra-

shard and a cross-shard transaction.

2.3.1 Network Sharding

- This is about splitting nodes into committees (node placement).
  - A node then only takes part in consensus and gossip for its own shard.
- Assignment uses randomness that can be verified: RandHound in OmniLedger [Kok+18], the Cuckoo rule in RapidChain [ZMR18], RANDAO in Ethereum [Nag+25]. The goal is to resist Sybil and adaptive attackers.
  - Elastico creates identities with PoW puzzles [Luu+16], while PoS designs use stake.
  - Reshuffling every epoch leaves an adaptive attacker little time to corrupt a committee, but every move forces nodes to download

the state of their new shard.

2.3.2 Transaction Sharding

- Transactions are usually sent to the shard of the sender or caller [Kok+18; ZMR18].
  - It is simple, everyone knows the rule, and clients need no extra information.
- Each shard has its own mempool. Other routing rules have been proposed but are uncommon.
  - Where a transaction goes only matters together with where its state lives: it is cheap only if all its state is in the same shard.

2.3.3 State Sharding

- Each shard keeps only part of the state. Where each account lives must be known by everyone and the load must be balanced.

  - Everyone needs to know where an account lives in order to route transactions to it.
  - No shard should become the bottleneck, either for storage or for processing.

- Most designs hash the account address modulo the number of shards: Chainspace [Al-+18], Monoxide [WW19], Elastico [Luu+16], and the Ethereum sharding proposals.
  - No metadata to store and a uniform spread, but it completely ignores which accounts talk to each other.

- Intra-shard vs. cross-shard transactions. Cross-shard ones need an atomic commit across shards, typically two-phase commit (lock, vote, commit or abort), which costs extra consensus rounds and messages [Kok+18].
  - OmniLedger (Atomix): the client locks inputs in each shard, then unlocks or commits [Kok+18].

  - RapidChain splits a cross-shard transaction into sub-transactions that go through the output shard [ZMR18].

  - Monoxide uses relay transactions and only gets eventual atomicity [WW19]. BrokerChain uses broker accounts whose state is spread over several shards [Hua+22].

  - Whatever the protocol, you pay extra latency and messages, state stays locked for a while, and some transactions abort.

- With hash placement, most Ethereum transactions become cross-shard. Part III gives the actual number.

  - Quick intuition: with k shards, two random accounts land in the same shard with probability $1/k$, and a transaction touching more accounts is almost never local.

2.3.4 Existing Sharded Blockchains and Their Limitations

- Go through the main designs: OmniLedger [Kok+18], RapidChain [ZMR18], Chainspace [Al-+18], Monoxide [WW19], Elastico [Luu+16], with Wang et al.'s survey [Wan+19] as a starting point. Then tell how Ethereum dropped execution sharding and moved to rollups plus data sharding [Pig+25].
  - Put them in a table: node placement, state placement, transaction placement, cross-shard protocol.
  - Some sharded chains are running outside Ethereum (NEAR, MultiversX). Mention them without going into detail.
- First problem: hash placement does not look at the workload at all, so the cross-shard ratio is very high [Kró+21].
- A lot of work tries to place state better and cuts the number of cross-shard transactions:

  - The idea comes from databases. Schism partitions tuples with METIS on the graph of what is accessed together [Cur+10; KK98], and Mehta et al. re-shard adaptively in an untrusted setting [Meh+25].
  - Graph-based placement: TxAllo [ZPY23], BrokerChain [Hua+22]. Community detection is also common.
  - Learning-based: Spring uses deep reinforcement learning [Li+24].
  - At the transaction level, OptChain picks a shard for each transaction to avoid cross-shard ones [Ngu+19].
  - They all report big drops in the cross-shard ratio on Ethereum traces.

- Second problem: they give an explicit location to *every* account. At Ethereum's scale that is hundreds of MB of metadata that every node must keep in sync.

- Third problem: a single coordinator decides the placement, or the paper

does not say who does. This opens the door to centralization and to people optimizing for themselves, and none of it is tied to consensus.

  - Usually nobody says who runs the optimizer, on what input, or how the others can check the result.
  - Whoever controls placement could favour their own accounts (cheaper, faster) or put victims in the same shard to attack them.

- Another line of work adjusts things at runtime, moving accounts around (Król et al. [Kró+21], Huang et al. [HLZ24], BIND [Qi+26]) or scheduling transactions together (Concordia [Liu+26]). They pay for it in migration or scheduling overhead.

  - Migration locks state while it moves [HLZ24]. BIND manages to keep processing transactions during the move [Qi+26].
  - Concordia has shards cooperate when scheduling to avoid cross-shard conflicts [Liu+26].

- What is missing is a placement that stays small, that anyone can check, and that is part of consensus. That is what Part III does.

2.4 Off-Chain Scaling: Layer 2

- The principle: run transactions off L1 but keep some or all of its security, using L1 for settlement and for publishing data [Gud+20].
  - Gudgeon et al. sort them into channels, sidechains/commit-chains and rollups [Gud+20].
  - A good way to compare them is to ask what an honest user needs to get their funds back if the operator cheats.
- L2s do not make L1 scaling unnecessary. They move the pressure onto publishing data on L1.
  - That is Ethereum's "rollup-centric" roadmap: L1 becomes the place where rollups settle and publish their data.

2.4.1 Rollups

- A sequencer orders and executes transactions off chain, then posts compressed transaction data and state roots to L1 [Gud+20].
  - A bridge contract on L1 holds the deposited funds and receives the state roots.
  - Today the sequencer is usually a single operator. Users can still force their transaction in through L1 if it censors them.
- Everything relies on the data being *available*: anyone must be able to rebuild the L2 state from it [Al-+21].
- Use one concrete example, Optimism (or Arbitrum [Kal+18]). Keep it short here and go into detail in Part IV.
  - In Optimism, the sequencer makes L2 blocks, a batcher posts them to L1 (as blobs now), and a proposer posts output roots. Anyone

can rebuild the L2 chain from L1 data alone.

  - A figure would help: L2 users, sequencer, batches/blobs on L1, verifiers rebuilding the state.

2.4.1.1 Optimistic Rollups

- State roots are accepted unless someone challenges them with a fraud proof during a challenge window of about 7 days [Kal+18; Gud+20].
  - In Arbitrum the two parties bisect the disputed execution until only one instruction is left, which L1 then checks [Kal+18].
  - The price of being optimistic is the long withdrawal delay.
- Challengers need the data to recompute the disputed state.
  - If the data is withheld, nobody can write a fraud proof, and an invalid root goes through.

2.4.1.2 Zero-Knowledge Rollups

- Each batch comes with a validity proof (SNARK/STARK) checked on L1. Finality is fast, but proving is expensive [Gud+20].
  - Proving EVM execution is costly, and zkEVMs give up some compatibility to bring that cost down.
- They still need data availability, so that users can rebuild the state and withdraw.
  - A valid proof tells you the new state is correct, not what it contains. Users still need the data to know what they own.

2.4.2 State Channels

- A fixed group of participants exchange updates off chain and settle on chain at the end, as in the Lightning Network [Gud+20].

  - The channel is opened on chain, signed state updates are exchanged off chain, and it is closed on chain. In a dispute, the latest signed state wins.
  - Payment networks route a payment across several channels.

- Limitations: funds are locked, participants have to stay online, and the set of participants is fixed, so channels are not general-purpose.

2.4.3 Sidechains

- These are separate chains with their own consensus, connected to L1 by a bridge (e.g., Polygon PoS) [Gud+20].
  - Two-way peg: lock assets on L1 to mint them on the sidechain, and the other way round.
- They do not get L1's security, and bridges have been hacked many times.

  - Funds are only as safe as the sidechain validators or the bridge committee.

- Use the comparison with rollups to explain why Ethereum's roadmap went with rollups.

2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations

- The data availability problem: how do you check that published data can be retrieved without downloading all of it [Al-+21]?

  - You cannot prove that someone withheld data, so you cannot slash for it. This is what makes it different from the other faults seen so far.

  - Light clients need availability guarantees for fraud proofs to be useful [Al-+21].

- EIP-4844 (blobs) [Pig+25] gives rollups their own cheaper, temporary space for data. But every node still downloads every blob, so capacity is capped by node bandwidth.
  - Blobs are large pieces of data attached to blocks, committed with KZG, not readable by the EVM, and deleted after a few weeks.
  - They have their own fee market and a small target and maximum number per block.
- Data availability sampling: erasure-code the data and let each node sample a few random pieces. This gives probabilistic guarantees while each node downloads only a small part (details in Part IV) [Al-+21; KZG10].
  - With erasure coding, any large enough fraction of the data is enough to rebuild all of it, so an attacker has to hide a lot to cause harm.

  - If that much is hidden, a few random samples are enough to notice with high probability.
  - Then explain the two ways to make sure the encoding is correct: fraud proofs [Al-+21], or KZG commitments, which rule out bad encoding by construction [KZG10].

- There are also separate DA layers (Celestia, EigenDA, Avail), with their own trust assumptions and timing.
  - A rollup that uses them no longer gets Ethereum's guarantees on its data.

- On the networking side, the options are GossipSub for dissemination and a DHT (Kademlia) for retrieval. PeerDAS is the first deployed step.
  - With gossip you need one topic per column or row. That does not scale well with many topics, and duplicate messages waste bandwidth.

  - DHT lookups take several hops, which is too slow for 4 s, and DHTs are exposed to Sybil and eclipse attacks.
  - PeerDAS has nodes keep a subset of columns and sample from their peers, but at a much smaller scale than the full design.

- What is missing: a way to spread and sample the data before the 4 s attestation deadline, at full Danksharding scale, while coping with faulty nodes and nodes that see different views of the network.

- This is what Part IV addresses [Pig+25].

Synthesis and Motivation 3

- A short chapter, a few pages, to close Part I: go from the limitations of Chapter 2 to the research questions.

3.1 Open Challenges

- C1: the designs above assume the workload has good locality, but this is rarely checked on real, recent data. This is Part II [PKR23].
  - Sharding, parallel execution and stateless clients are mostly evaluated on synthetic or short traces.
  - The question: can solid assumptions about the near future be drawn from historical data?
- C2: state placement has to cut cross-shard transactions while keeping

its metadata small, staying verifiable, and not letting anyone tilt it in their favour. This is Part III.

  - It should fit into the existing consensus and its safeguards, without a trusted coordinator.

- C3: L2 data availability has to be checked by sampling, within consensus timing, over an open P2P network. This is Part IV.

  - The constraints: a hard 4 s deadline, tens of thousands of nodes, some of them faulty or unresponsive, and no new consensus step.

3.2 Common Requirements for L1 and L2 Sharding

- Verifiable: any node must be able to recompute who is assigned what (accounts to shards, cells to nodes), with no trusted third party.

  - This uses the same trick as Ethereum committees: deterministic functions of public randomness and public state (Section 1.3.3).

- Flexible: able to adapt in large, changing systems (the workload evolves, nodes join and leave, some fail).
  - Fully deterministic schemes are too rigid, and fully ad-hoc ones cannot be checked. My designs try to sit in between.
- Decentralized: no heavy new consensus step, low cost per node, anyone can participate.
- Practical: evaluated on real Ethereum workloads and timing, and compatible with the existing protocols.

3.3 Positioning of the Contributions

- SOAPS shards computation on L1. It removes contention on execution and state, and it benefits from knowing the workload (Part II).
- PANDAS [Pig+25] shards the L2 data. It removes the bandwidth bottleneck for data availability, and it barely depends on the workload.

- The two complement each other rather than compete: a sharded L1 still has to host L2 data, and an L1 full of rollups still gains from parallel execution.

- A figure would help: L1 vs. L2 on one axis, computation vs. data on the other, with Parts II to IV placed on it.

- End with a short roadmap of Parts II to IV.

Bibliography

[Al-+18] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis. "Chainspace: A Sharded Smart Contracts Platform". In: *NDSS*. 2018.

[Al-+21] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi. "Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2021, pp. 279–298.

[And+18] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, et al. "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains". In: *ACM EuroSys*. 2018.

[Ava+23] Z. Avarikioti, A. Desjardins, L. Kokoris-Kogias, and R. Wattenhofer. "Divide & scale: Formalization and roadmap to robust sharding". In: *International Colloquium on Structural Information and Communication Complexity*. Springer. 2023, pp. 199–245.

[Bla+24] S. Blackshear et al. "Sui Lutris: A Blockchain Combining Broadcast and Consensus". In: *ACM CCS*. 2024, pp. 2606–2620.

[BLS01] D. Boneh, B. Lynn, and H. Shacham. "Short Signatures from the Weil Pairing". In: *Advances in Cryptology (ASIACRYPT)*. 2001, pp. 514–532.

[CBC22] P. Chatzigiannis, F. Baldimtsi, and K. Chalkias. "SoK: Blockchain Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 615–641.

[CL99] M. Castro and B. Liskov. "Practical Byzantine Fault Tolerance". In: *USENIX OSDI*. 1999, pp. 173–186.

[Cur+10] C. Curino, E. P. C. Jones, Y. Zhang, and S. R. Madden. "Schism: a workload-driven approach to database replication and partitioning". In: *Proc. VLDB Endow.* (2010).

[Dai+20] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability". In: *IEEE S&P*. 2020, pp. 910–927.

[Dem+87] A. Demers, D. Greene, C. Hauser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, and D. Terry. "Epidemic Algorithms for Replicated Database Maintenance". In: *ACM PODC*. 1987, pp. 1–12.

[Dic+17] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen. "Adding Concurrency to Smart Contracts". In: *ACM PODC*. 2017, pp. 303–312.

[DLS88] C. Dwork, N. Lynch, and L. Stockmeyer. “Consensus in the Presence of Partial Synchrony”. In: *Journal of the ACM* 35.2 (1988), pp. 288–323.

[DN92] C. Dwork and M. Naor. “Pricing via Processing or Combatting Junk Mail”. In: *Advances in Cryptology (CRYPTO)*. 1992, pp. 139–147.

[Dou02] J. R. Douceur. “The Sybil Attack”. In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 251–260.

[Eth] Ethereum Foundation. *Ethereum Proof-of-Stake Consensus Specifications*. URL: `https://github.com/ethereum/consensus-specs` (visited on 10/12/2026).

[FLP85] M. J. Fischer, N. A. Lynch, and M. S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process”. In: *J. ACM* 32.2 (1985), pp. 374–382.

[GDW19] D. Guo, J. Dong, and K. Wang. "Graph structure and statistical properties of Ethereum transaction relationships". In: *Inf. Sci.* 492 (2019), pp. 58–71.

[Gel+23] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou. "Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing". In: *ACM PPoPP*. 2023, pp. 232–244.

[Ger+16] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *ACM CCS*. 2016, pp. 3–16.

[Gil+17] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. "Algorand: Scaling Byzantine Agreements for Cryptocurrencies". In: *ACM SOSP*. 2017, pp. 51–68.

[GKL15] J. Garay, A. Kiayias, and N. Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *EUROCRYPT*. 2015, pp. 281–310.

[Gud+20] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais. "SoK: Layer-Two Blockchain Protocols". In: *Financial Cryptography and Data Security (FC)*. 2020, pp. 201–226.

[Hei+23] L. Heimbach, L. Kiffer, C. Ferreira Torres, and R. Wattenhofer. "Ethereum's Proposer-Builder Separation: Promises and Realities". In: *ACM IMC*. 2023, pp. 406–420.

[HLZ24] H. Huang, Y. Lin, and Z. Zheng. "Account migration across blockchain shards using fine-tuned lock mechanism". In: *IEEE INFOCOM 2024*. 2024, pp. 271–280.

[HS91] S. Haber and W. S. Stornetta. "How to Time-Stamp a Digital Document". In: *Journal of Cryptology* 3.2 (1991), pp. 99–111.

[Hua+22] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo. "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding". In: *IEEE INFOCOM*. 2022, pp. 1968–1977.

[Kal+18] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten. "Arbitrum: Scalable, private smart contracts". In: *USENIX Security*. 2018, pp. 1353–1370.

[Kia+17] A. Kiayias, A. Russell, B. David, and R. Oliynykov. "Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol". In: *CRYPTO*. 2017, pp. 357–388.

[KK98] G. Karypis and V. Kumar. "A fast and high quality multilevel scheme for partitioning irregular graphs". In: *SIAM J. Sci. Comput.* 20.1 (1998), pp. 359–392.

[Kok+18] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford. “OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding”. In: *IEEE S&P*. 2018, pp. 583–598.

[Kró+21] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière. “Shard scheduler: Object placement and migration in sharded account-based blockchains”. In: *ACM AFT*. 2021, pp. 43–56.

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *ASIACRYPT*. 2010, pp. 177–194.

[Li+24] P. Li, M. Song, M. Xing, Z. Xiao, Q. Ding, S. Guan, and J. Long. “Spring: Improving the throughput of sharding blockchain via deep reinforcement learning based state placement”. In: *ACM Web Conf*. 2024, pp. 2836–2846.

[Liu+26] Y. Liu, L. Jia, X. Yang, Z. Li, and Y. Sun. "Concordia: Enabling Low-Conflict Distributed Transaction Scheduling in Sharding Blockchain via Cooperative Perception". In: *ACM Web Conf.* 2026, pp. 5493–5502.

[LSP82] L. Lamport, R. Shostak, and M. Pease. "The Byzantine Generals Problem". In: *ACM Trans. Program. Lang. Syst.* 4.3 (1982), pp. 382–401.

[Luu+16] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena. "A Secure Sharding Protocol For Open Blockchains". In: *ACM CCS.* 2016, pp. 17–30.

[Meh+25] B. Mehta, N. Baghel, M. J. Amiri, B. T. Loo, and R. Marcus. "Adaptive Sharding in Untrusted Environments". In: *Proc. ACM Manag. Data* 3.6 (2025), pp. 1–28.

[Mer87] R. C. Merkle. “A Digital Signature Based on a Conventional Encryption Function”. In: *Advances in Cryptology (CRYPTO)*. 1987, pp. 369–378.

[MM02] P. Maymounkov and D. Mazières. “Kademlia: A Peer-to-Peer Information System Based on the XOR Metric”. In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 53–65.

[Nag+25] Á. Nagy, J. Tapolcai, I. A. Seres, and B. Ladóczki. “Forking the RANDAO: Manipulating Ethereum’s Distributed Randomness Beacon”. In: *ACM CCS*. 2025, pp. 873–887.

[Nak08] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. `https://bitcoin.org/bitcoin.pdf`. 2008.

[Ngu+19] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai. “OptChain: Optimal Transactions Placement for Scalable Blockchain Sharding”. In: *IEEE ICDCS*. 2019, pp. 525–535.

[NTT21] J. Neu, E. N. Tas, and D. Tse. "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma". In: *IEEE S&P*. 2021, pp. 446–465.

[Pac+23] M. Pacheco, G. Oliva, G. K. Rajbahadur, and A. Hassan. "Is my transaction done yet? an empirical study of transaction processing times in the ethereum blockchain platform". In: *ACM Trans. Softw. Eng. Methodol.* 32.3 (2023), pp. 1–46.

[PAT24] U. Pavloff, Y. Amoussou-Guenou, and S. Tucci-Piergiovanni. "Byzantine attacks exploiting penalties in Ethereum PoS". In: *IEEE/IFIP DSN*. 2024, pp. 53–65.

[Pig+25] M. Pigaglio, O. Ascigil, M. Król, F. Lange, K. Peeroo, S. Rene, R. Sadre, V. Stankovic, and E. Rivière. "PANDAS: Peer-to-peer, Adaptive Networking Allowing Data Availability Sampling within

Ethereum Consensus Timebounds". In: *Middleware*. 2025, pp. 167–179.

[PKR23] M. Pigaglio, M. Król, and E. Rivière. "Exploring locality in Ethereum transactions". In: *IEEE BRAINS*. 2023, pp. 1–8.

[Qi+26] J. Qi, D. Ding, J. Li, J. Cao, Y.-C. Chen, G. Xue, and S. Liu. "BIND: Enabling Continuous Transaction Processing During Account Migration in Sharded Blockchains". In: *ACM Web Conf.* 2026, pp. 5547–5556.

[Rou24] T. Roughgarden. "Transaction Fee Mechanism Design". In: *J. ACM* (2024), pp. 1–25.

[Sch+22] C. Schwarz-Schilling, J. Neu, B. Monnot, A. Asgaonkar, E. N. Tas, and D. Tse. "Three Attacks on Proof-of-Stake Ethereum". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 560–576.

[Sch90] F. B. Schneider. “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial”. In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319.

[She+21] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath. “BFT Protocol Forensics”. In: *ACM CCS*. 2021, pp. 1722–1743.

[SZ15] Y. Sompolinsky and A. Zohar. “Secure High-Rate Transaction Processing in Bitcoin”. In: *Financial Cryptography and Data Security (FC)*. 2015, pp. 507–527.

[Wan+19] G. Wang, Z. J. Shi, M. Nixon, and S. Han. “SoK: Sharding on Blockchain”. In: *ACM AFT*. 2019, pp. 41–61.

[Woo14] G. Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Ethereum Yellow Paper, `https://ethereum.github.io/yellowpaper/paper.pdf`. 2014.

[WW19] J. Wang and H. Wang. “Monoxide: Scale out blockchains with asynchronous consensus zones”. In: *USENIX NSDI*. 2019, pp. 95–112.

[Yin+19] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. “HotStuff: BFT Consensus with Linearity and Responsiveness”. In: *ACM PODC*. 2019, pp. 347–356.

[ZMR18] M. Zamani, M. Movahedi, and M. Raykova. “RapidChain: Scaling Blockchain via Full Sharding”. In: *ACM CCS*. 2018, pp. 931–948.

[ZPY23] Y. Zhang, S. Pan, and J. Yu. “TxAllo: Dynamic Transaction Allocation in Sharded Blockchain Systems”. In: *IEEE ICDE*. 2023, pp. 721–733.